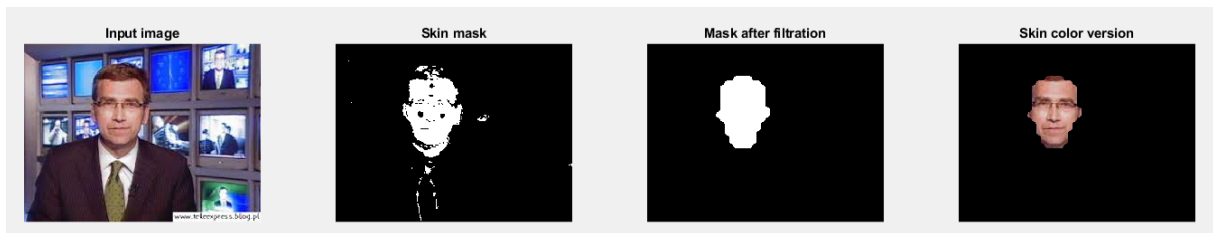


Human-Computer Interaction

Camera mouse (3): face (or hand) mouse

In the last class, we created a simple colour detector using the example of a face detector (based on the skin color technique).



In this class, we will extend that idea by adding a camera input and using it to track the position of the face (or hand) in real time.

Finally, we will map the detected head (or hand) movement to control the position of the mouse pointer on the screen.

1. Calculate the centroid (center of mass/gravity) of the mask after filtration:



We can calculate the centroid on the basis of the method of moments.

$$\begin{cases} x_c = \frac{M_{10}}{M_{00}} \\ y_c = \frac{M_{01}}{M_{00}} \end{cases}$$

$$M_{10} = \sum_{x=x_1}^{x_2} \sum_{y=y_1}^{y_2} x I(x, y)$$

$$M_{01} = \sum_{x=x_1}^{x_2} \sum_{y=y_1}^{y_2} y I(x, y)$$

$$M_{00} = \sum_{x=x_1}^{x_2} \sum_{y=y_1}^{y_2} I(x, y)$$

Hint: You can easily compute the center of gravity using OpenCV. The function `cv2.moments()` returns all image moments for a given contour or binary mask.

Once you have them, you can find the centroid as:

```
M = cv2.moments(mask)
cx = int(M["m10"] / M["m00"])
cy = int(M["m01"] / M["m00"])
```

2. Analyse the camera code

Another element we will add is the camera. The sample code below captures an image, flips it horizontally, and displays it.

```
1  import cv2
2
3  # Open the default camera (0 = first connected camera)
4  cap = cv2.VideoCapture(0)
5
6  if not cap.isOpened():
7      print("Cannot open camera.")
8      exit()
9
10 while True:
11     # Read one frame from the camera
12     ret, frame = cap.read()
13     if not ret:
14         print("Failed to grab frame.")
15         break
16
17     # Flip the frame horizontally (mirror effect)
18     mirrored = frame[:, ::-1] # Alternative method: mirrored = cv2.flip(frame, 1)
19
20     # Show the mirrored frame
21     cv2.imshow("Camera - mirrored (press 'q' to quit)", mirrored)
22
23     # Exit the loop when 'q' is pressed
24     if cv2.waitKey(1) & 0xFF == ord('q'):
25         break
26
27 # Release the camera and close all OpenCV windows
28 cap.release()
29 cv2.destroyAllWindows()
```

3. Combine skin color detector and centroid calculation with the camera.

4. Finally, based on the calculated coordinates of the face (or hand) center, use the head (or hand) movement to steer the computer mouse.

To map head movement to the full screen, you need the dimensions of the display - get the screen size.

There are multiple Python libraries you can use: `screeninfo`, `pyautogui`, `tkinter`.

```
from screeninfo import get_monitors

monitor = get_monitors()[0] # assuming primary monitor
screen_width = monitor.width
screen_height = monitor.height

print(f"Screen width: {screen_width}, height: {screen_height}")
```

```
import pyautogui
screen_w, screen_h = pyautogui.size()
```

Map the face coordinates (center of gravity) to the screen dimensions (new mouse position)

```
mouse_x = int((cx / w) * screen_width)
mouse_y = int((cy / h) * screen_height)
```

Move the mouse based on calculated coordinates. You can use multiple Python libraries: pyautogui, pynput, mouse, ctypes (Windows)

```
import pyautogui
pyautogui.moveTo(mouse_x, mouse_y)
```

```
from pynput.mouse import Controller
mouse = Controller()
mouse.position = (mouse_x, mouse_y)
```

5. Additional Tasks – For Ambitious Students

Reduce jittery mouse movement by averaging several consecutive face positions.

This helps stabilize the cursor and prevents small head shakes from causing rapid jumps.

Hint: You can store a few recent face positions and calculate their average:

```
avg_x = sum(last_x_positions) / len(last_x_positions)
```

```
avg_y = sum(last_y_positions) / len(last_y_positions)
```

Apply a movement scaling factor

Control how much the cursor moves in response to head movement.

By adjusting a scaling factor, you can make the cursor more or less sensitive to small changes in position.

Hint: You can smooth the movement using interpolation:

```
scaled_x = prev_x + (current_x - prev_x) * 0.3
```

```
scaled_y = prev_y + (current_y - prev_y) * 0.3
```

Calibrate the neutral head position

Add a short calibration step that records the initial face position and treats it as the screen center.

This will allow more natural and comfortable control relative to the user's resting position.

Report

- Prepare a report on the camera mouse problem in PDF format.
- Send the report to: anowosielski@zut.edu.pl
 - In the e-mail title, please use the scheme: HCI Your Full Name